

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: Milestone 2 API Testing

Student: Deep S. (dns33)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/13/2026 9:22:25 PM

Updated: 4/16/2026 6:58:17 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/milestone-2-api-testing/grading/dns33>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/milestone-2-api-testing/view/dns33>

Instructions

• Overview Link: <https://youtu.be/86v8Ak0E3XE>

1. Ensure you read all instructions and objectives before starting.
2. Ensure you added your API choice to the Google doc shared on Canvas (selections must be unique, no two students can use the same exact API)
3. Create a new branch from Milestone2 called M7-Homework
 1. `git checkout Milestone2` (ensure proper starting branch)
 2. If you don't have Milestone2 yet, create it from Milestone1
 3. `git pull origin Milestone2` (ensure history is up to date)
 4. `git checkout -b M7-Homework` (create and switch to branch)
4. Follow the lesson from
 - Add `testApi.php` to your project folder
 - Immediately record to history
 - ☐ `git add public_html`
 - ☐ `git commit -m "adding M7 HW baseline files"`
 - ☐ `git push origin M7-Homework`
 - ☐ Create a Pull Request from M7-Homework to Milestone2 and keep it open
5. Fill out the below worksheet
 - Ensure you add the UCID/Date and details comments as you work
 - Get the example working with your chosen API
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin M7-Homework`
 4. On Github merge the pull request from M7-Homework to Milestone2
 5. On Github create and merge the pull request from Milestone2 to qa
 6. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
8. Upload the same PDF to Canvas
9. Sync Local

1. git checkout qa
2. git pull origin qa
3. git checkout Milestone2
4. git pull origin Milestone2

Section #1: (2 pts.) Analyze Api Choice

Progress: 100%

Task #1 (0.40 pts.) - API Link

Progress: 100%

URL #1

<https://rapidapi.com/h0p3rwe/api/youtube-search-and-download>



URL

<https://rapidapi.com/h0p3rwe/api/youtube-search-and-download>



Saved: 4/13/2026 9:45:48 PM

Task #2 (0.80 pts.) - Quota and Limitations

Progress: 100%

Details:

- Note: Hard limits stop when reached, soft limits incur costs when reached (and need a valid credit card associated)
- It's recommend to use a free API with hard limits over one with soft limits, but be mindful of the quota value as it's easy to waste it (through the site's dashboard and/or your own code)

Your Response:

API name is Youtube Search and Download This API is on the Basic (\$0/month) plan. It has a hard limit of 500 requests per day, meaning once I hit 500 requests in a day, the API will stop responding until the quota resets (no extra charges). It also has a rate limit of 1000 requests per hour, which prevents sending too many requests too quickly.

The plan includes 10,240 MB (10 GB) of bandwidth per month. If the bandwidth limit is exceeded, RapidAPI may charge \$0.001 per additional 1 MB. To avoid wasting quota/bandwidth, I will minimize unnecessary refreshes/testing, cache results when possible, and avoid repeatedly calling the API in loops.



Saved: 4/13/2026 9:48:24 PM

Task #3 (0.80 pts.) - Goal Data

Progress: 100%

Details:

- Clearly note what type of data you'll be using
- Clearly note the specific properties you intend to use (it's likely not all of them)

Your Response:

Type of data I'll be using: I will use YouTube video search results returned by the Youtube Search and Download API. The response contains a list of video objects under contents[].

Specific properties I intend to use: From each contents[i].video object, I will use: • videoid (unique video identifier) • title (video title) • channelName and channelId (who posted the video) • publishedTimeText (when it was posted) • lengthText (duration) • viewCountText (view count text) • thumbnails[n].url (thumbnail image URL to display a preview)



Saved: 4/13/2026 10:02:39 PM

Section #2: (6 pts.) Verify Api Choice

Progress: 100%

≡ Task #1 (6 pts.) - Edit the `testApi.php` file (and related `.env` content) to test your chosen API

Progress: 100%

Details:

- The code will need to be adjusted since your API differs from the one I chose
- You'll need to apply the API .env items to Render.com's Environment variables both on qa and prod
- Be mindful of your API quota (may need to initially use a cached response until you get it working, refer to the lessons for this)

Part 1:

Progress: 100%

Details:

Two screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) 2. Don't show any private details like the API key
2. Full output of executing the testApi.php (visit the proper file on Render.com qa after a manual deploy)

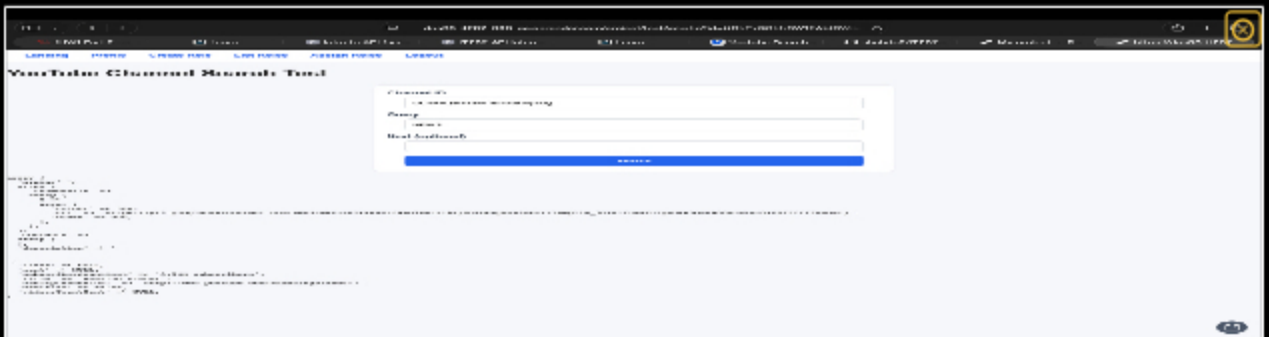


```
1 <?php
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

snippet 1

```
1 <?php
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

snippet 2



output screenshot

 Saved: 4/16/2026 6:53:20 PM

Part 2:

Progress: 100%

Details:

- Direct link to the file in the homework related branch from Github (should end in .php , zero credit if it does not)
- Direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)

URL #1

https://github.com/shahdp2/IT202-SP2026/blob/M7-Homework/public_html/project/testApi.php



URL

<https://github.com/shahdp2/IT202>



URL #2

<https://dns33-it202-008-prod.onrender.com/project/testApi.php>



URL

<https://dns33-it202-008-prod.onrender.com/project/testApi.php>



Progress: 100%

Details:

Briefly explain how your solution works. Describe the logic, steps, or decisions your code makes to reach the result. Do not restate the problem or list requirements.

Your Response:

My solution reads the RapidAPI key from environment variables (or a local .env file when running locally) so the secret is never hardcoded in the PHP file. When the page loads, it checks whether the required query parameters are present (the channel id and a query, with next as optional pagination). If they are present, it builds a \$data array containing those values and sends a GET request to the YouTube Search & Download endpoint using my reusable get() helper.

The `get()` helper formats the request by attaching the query parameters to the URL and adding the RapidAPI headers (`X-RapidAPI-Key` and `X-RapidAPI-Host`). It executes the request with `cURL` and returns a structured result containing an HTTP status and the raw response body. Back in `testApi-
yt.php`, I log the raw response to Render logs for debugging, then only decode the JSON when the request succeeds. Finally, I display the decoded response (or an error payload if something fails), which confirms the API call worked and shows the returned fields from the endpoint.

Section #3: (2 pts.) Misc

Progress: 100%

Task #1 (0.67 pts.) - Github Details

Progress: 100%

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present

The screenshot shows the GitHub interface for the 'magento2/magento2' repository. The 'Commits' tab is selected, displaying a list of recent commits. The commits are ordered by date, with the most recent at the top. Each commit entry includes a green checkmark icon, a commit message, the author's name, the date, and the commit hash. The 'Merge pull request' commits are highlighted with a green 'Merged' label. The 'Merge pull request' commits are: 'Merge pull request #4075 from shivakp/magento2-4075' and 'Merge pull request #4074 from shivakp/magento2-4074'. The 'Merge pull request' commits are highlighted with a green 'Merged' label.



Saved: 4/16/2026 6:56:38 PM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in /pull/#)

URL #1

<https://github.com/shahdp2/IT202-SP2026/pull/82>



URL

<https://github.com/shahdp2/IT20>



Saved: 4/16/2026 6:56:38 PM

 Task #2 (0.67 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the [WakaTime.com](https://wakaTime.com) Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waketime



Saved: 4/16/2026 6:55:55 PM

≡ Task #3 (0.67 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to integrate a third-party API into a PHP project using RapidAPI and cURL while keeping secrets safe. I also learned how to structure requests with query parameters, required headers, and how to decode and display JSON results. Another important takeaway was using environment variables (.env locally + Render env vars) so API keys are not exposed in code or screenshots, and using logs to debug responses safely.



Saved: 4/16/2026 6:58:01 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was getting a basic "test page" working once I understood the endpoint and the parameters it needed. After I confirmed the request worked in RapidAPI's test console, translating that into PHP was straightforward. Displaying the output with `var_export()` and confirming the response fields also made it easy to verify success.



Saved: 4/16/2026 6:58:08 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was troubleshooting issues caused by small setup mistakes, like missing includes (e.g., `get()` not being loaded) or incorrect environment variable

setup between local and Render. It also took time to make sure I wasn't leaking private details (API keys) in code snippets or screenshots while still showing proof that the API call works. Debugging with logs and carefully validating what gets printed to the screen was the key challenge.



Saved: 4/16/2026 6:58:17 PM